



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 07/10

APIs et Communication

REST + HTTP + JSON + Endpoints + Inter-services

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - C'est quoi HTTP ?	3
Chapitre 2 - Les methodes HTTP	6
Chapitre 3 - Les codes de statut	9
Chapitre 4 - Headers HTTP	12
Chapitre 5 - JSON en details	15
Chapitre 6 - REST : principes	18
Chapitre 7 - Endpoints du chatbot-service	21
Chapitre 8 - Endpoints de l'academic-service	26
Chapitre 9 - Endpoints LLM	31
Chapitre 10 - Communication inter-services	34
Chapitre 11 - Proxy Angular (proxy.conf.json)	37
Chapitre 12 - Swagger/OpenAPI	40
Chapitre 13 - Tester les APIs	43
Chapitre 14 - Conclusion	46

Chapitre 1 - C'est quoi HTTP ?

1.1 Definition

HTTP = HyperText Transfer Protocol. C'est le **langage** que parlent les navigateurs et serveurs web depuis 1991. Chaque fois que tu visites un site web, ton navigateur fait une (ou plusieurs) requete HTTP.

■ **ANALOGIE :** HTTP est comme la **langue parlée** dans un magasin. Le client (navigateur) parle au vendeur (serveur) selon une grammaire stricte : 'Bonjour, je voudrais X, voici mon ID...' Le serveur repond pareil : 'Voici X, ca coute Y, merci'.

1.2 Caracteristiques cles

- **Stateless** : chaque requete est independante (pas de memoire)
- **Client-serveur** : le client demande, le serveur repond
- **Request-Response** : pas de communication initiee par le serveur
- **Texte** : protocole texte (lisible), mais peut transporter du binaire
- **Sur TCP** : utilise le protocole TCP/IP
- **HTTPS** : version chiffree (TLS/SSL), securise

1.3 Anatomie d'une requete HTTP

```
POST /api/chat HTTP/1.1
Host: localhost:8001
Content-Type: application/json
Authorization: Bearer eyJ0eXAiOiJKV1Qi...
User-Agent: Mozilla/5.0 (Windows NT 10.0) Chrome/120.0

{
  "message": "Quelles sont les filieres ?",
  "session_id": "sess_abc123"
}
```

Structure :

- **Premiere ligne** : methode + URL + version protocole
- **Headers** : metadonnees (type contenu, auth, etc.)
- **Ligne vide** : separe headers et body
- **Body** : donnees envoyees (optionnel, surtout pour POST/PUT)

1.4 Anatomie d'une reponse HTTP

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 245
```

Date: Wed, 28 May 2026 13:24:35 GMT

```
{  
  "response": "La FSBM propose 7 licences...",  
  "intent": "filieres",  
  "confidence": 1.0,  
  "session_id": "sess_abc123"  
}
```

Chapitre 2 - Les methodes HTTP

2.1 Les 5 principales

Methode	But	Idempotent	Body
GET	Lire une ressource	Oui	Non
POST	Creer une ressource	Non	Oui
PUT	Remplacer integrelement	Oui	Oui
PATCH	Modifier partiellement	Non	Oui
DELETE	Supprimer	Oui	Non

Idempotent : appeler la methode plusieurs fois donne le meme resultat. GET, PUT, DELETE le sont. POST ne l'est PAS (2 POST = 2 ressources creees).

2.2 GET

Pour lire des donnees. **Pas de body**, les parametres sont dans l'URL.

```
# Lecture simple
GET /api/filieres

# Avec filtres en query string
GET /api/filieres?type=MASTER&search=info

# Avec parametre de chemin
GET /api/filieres/SMI

# En Angular
this.http.get<Filiere[]>('/api/filieres');
```

2.3 POST

Pour creer ou pour declencher une action complexe. **Body JSON** contient les donnees.

```
POST /api/chat
Content-Type: application/json

{
  "message": "Bonjour",
  "session_id": "sess_abc"
}

# En Angular
this.http.post('/api/chat', {
  message: 'Bonjour',
  session_id: 'sess_abc'
});
```

2.4 PUT vs PATCH

```
# PUT - remplace TOUT le profil
PUT /api/users/42
{ "name": "Karim", "email": "karim@etu.fsbm.ma", "age": 21 }
# Si on omet un champ, il est mis a default/null

# PATCH - modifie SEULEMENT certains champs
PATCH /api/users/42
{ "email": "new@etu.fsbm.ma" }
# Seul email change, le reste est intact
```

2.5 DELETE

```
DELETE /api/messages/123
# Pas de body, juste l'URL

# En Angular
this.http.delete('/api/messages/123');
```

Chapitre 3 - Les codes de statut

3.1 Les 5 categories

Plage	Categorie	Signification
1xx	Informationnel	Requete recue, traitement en cours
2xx	Succes	Requete reussie
3xx	Redirection	Action complementaire necessaire
4xx	Erreur client	Le client a fait une erreur
5xx	Erreur serveur	Le serveur a un probleme

3.2 Les codes essentiels

Code	Nom	Quand le retourner
200	OK	Succes general (GET, PUT, PATCH)
201	Created	POST qui a cree une ressource
204	No Content	Succes sans donnees (DELETE)
301	Moved Permanently	Redirection definitive
302	Found	Redirection temporaire
304	Not Modified	Cache du client encore valide
400	Bad Request	Mauvais format (JSON invalide)
401	Unauthorized	Pas d'auth ou token expire
403	Forbidden	Auth OK mais pas les droits
404	Not Found	Ressource introuvable
409	Conflict	Ex: email deja utilise
422	Unprocessable	Validation Pydantic echouee
429	Too Many	Rate limit atteint
500	Internal Server Error	Bug serveur
502	Bad Gateway	Service en aval down
503	Service Unavailable	Surcharge

3.3 Exemple en FastAPI

```
@router.get('/api/filieres/{code}')
async def get_filiere(code: str, db = Depends(get_db)):
    f = await db.find_by_code(code)
    if not f:
        raise HTTPException(
```

```
        status_code=404,  
        detail=f'Filiere {code} introuvable'  
    )  
    return f # 200 OK
```


Chapitre 4 - Headers HTTP

4.1 Headers de requete courants

Header	Role	Exemple
Content-Type	Type du body	application/json
Accept	Type attendu en reponse	application/json
Authorization	Token d'auth	Bearer eyJ0eXAi...
User-Agent	Identifie le client	Mozilla/5.0...
Accept-Language	Langue preferee	fr-FR,fr;q=0.9
Cookie	Cookies	session_id=abc...
Origin	Domaine d'origine (CORS)	http://localhost:4200
Referer	Page d'origine	http://example.com/page

4.2 Headers de reponse courants

Header	Role
Content-Type	Type du body retourne
Content-Length	Taille du body en bytes
Set-Cookie	Definir un cookie
Cache-Control	Politique de cache
Access-Control-Allow-Origin	CORS (autorise quel domaine)
Location	Pour les redirections
X-RateLimit-Remaining	Requetes restantes

4.3 CORS expliqué

CORS = Cross-Origin Resource Sharing. Par default, le navigateur empeche un site (localhost:4200) de faire des requetes vers un autre (localhost:8001). C'est pour la securite. Le backend doit **explicitement autoriser** les origines.

```
# FastAPI CORS
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:4200'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)
```

☐ Si tu vois 'CORS error' dans la console Angular, c'est que ton frontend n'est pas dans la whitelist du backend.

Chapitre 5 - JSON en details

5.1 Qu'est-ce que JSON ?

JSON = JavaScript Object Notation. Format de donnees structurees, lisible par humain et facile a parser pour les machines. Standard de facto pour les APIs REST.

5.2 Syntaxe

```
{
  "string": "texte entre guillemets doubles",
  "number": 42,
  "float": 3.14,
  "boolean": true,
  "null_value": null,
  "array": [1, 2, 3, "melange autorise"],
  "object": {
    "nested": "sous-objet"
  }
}
```

5.3 Regles strictes

- Cles TOUJOURS entre **guillemets doubles**
- Pas de guillemets simples (contrairement a Python)
- Pas de commentaires // ou /* */
- Pas de virgule finale apres le dernier element
- Encodage UTF-8 obligatoire

5.4 Exemple FSBM : une filiere

```
{
  "id": 10,
  "code": "IADS",
  "name": "Master Intelligence Artificielle et Data Science",
  "type": "MASTER",
  "department_id": 1,
  "coordinator": "Pr. Mehdi FILALI",
  "coord_email": "m.filali@fsbm.ac.ma",
  "duration_years": 2,
  "capacity": 25,
  "description": "Machine learning, deep learning, NLP...",
  "careers": "Data scientist, ingénieur ML",
  "is_active": true,
  "created_at": "2026-01-15T10:30:00Z"
}
```

5.5 Parser JSON

```
# Python
import json
data = json.loads('{"name": "Karim"}')
print(data['name']) # Karim

# JavaScript / TypeScript
const data = JSON.parse('{"name": "Karim"}');
console.log(data.name); // Karim

// Avec HttpClient Angular, parsing auto
this.http.get<Filiere>('/api/filieres/SMI')
  .subscribe(f => console.log(f.name));
```

Chapitre 6 - REST : principes

6.1 Qu'est-ce que REST ?

REST = Representational State Transfer. Style architectural defini par Roy Fielding (2000). Ce n'est PAS un standard ni un protocole, mais des principes a respecter pour designer des APIs.

6.2 Les 6 principes

1. **Client-Server.** Separation stricte entre client (UI) et serveur (logique + donnees).
2. **Stateless.** Chaque requete contient toute l'info necessaire. Pas de session cote serveur (le JWT est dans le header).
3. **Cacheable.** Les reponses indiquent si elles peuvent etre mises en cache.
4. **Uniform Interface.** URLs identifient les ressources, methodes HTTP indiquent l'action.
5. **Layered System.** Plusieurs couches possibles (load balancer, cache, etc.).
6. **Code on demand (optionnel).** Le serveur peut envoyer du code executable (JavaScript).

6.3 Convention de nommage

Action	URL	Methode
Lister	/api/filieres	GET
Detail	/api/filieres/SMI	GET
Creer	/api/filieres	POST
Modifier (tout)	/api/filieres/1	PUT
Modifier (partiel)	/api/filieres/1	PATCH
Supprimer	/api/filieres/1	DELETE
Sous-ressources	/api/filieres/1/modules	GET

6.4 Bonnes pratiques

- + URLs en **kebab-case** ou **snake_case**, pas en CamelCase
- + Pluriel pour les collections : /filieres pas /filiere
- + Verbe non recommande dans l'URL (la methode HTTP suffit)
- + Pagination : ?page=1&page_size=20
- + Tri : ?sort=name■=asc
- + Filtrage : ?type=MASTER&search;=info
- + Versionning : /api/v1/... si breaking changes

Chapitre 7 - Endpoints du chatbot-service

7.1 POST /api/chat

```
Request :
POST /api/chat
Content-Type: application/json
{
  "message": "Quelles sont les filieres ?",
  "session_id": "sess_abc123",
  "user_id": null,
  "language": null
}

Response :
200 OK
{
  "response": "La FSBM propose 7 licences...",
  "intent": "filieres",
  "confidence": 1.0,
  "conversation_id": 42,
  "session_id": "sess_abc123",
  "language": "fr",
  "language_confidence": 1.0,
  "top_candidates": [...],
  "suggestions": ["Master IADS", ...],
  "news_items": [],
  "response_time_ms": 152
}
```

7.2 POST /api/chat/feedback

```
Request :
POST /api/chat/feedback
{
  "conversation_id": 42,
  "note": 5,
  "is_helpful": true,
  "commentaire": "Tres clair !"
}

Response :
200 OK
{
  "success": true,
  "feedback_id": 123
}
```

7.3 GET /api/chat/history/{session_id}

```
Request :
GET /api/chat/history/sess_abc123

Response :
200 OK
{
```

```

"session_id": "sess_abc123",
"total_messages": 4,
"history": [
  { "sender": "user", "text": "Bonjour", "timestamp": "..."},
  { "sender": "bot", "text": "Bonjour !", "intent": "salutation"},
  ...
]
}

```

7.4 GET /api/chat/suggestions

```

Request :
GET /api/chat/suggestions?lang=fr&limit=6

Response :
{
  "language": "fr",
  "total": 6,
  "suggestions": [
    { "intent": "inscription", "icon": "■",
      "question": "Comment s'inscrire a la FSBM ?" },
    ...
  ]
}

```

7.5 GET /api/chat/news

```

Request :
GET /api/chat/news?source=all&limit=8&lang=fr

Response :
{
  "fetched_at": "2026-05-28T...",
  "total": 6,
  "sources": {
    "local_db": { "ok": true, "count": 5 },
    "fsbm.ma": { "ok": true, "count": 0 },
    "facebook": { "ok": true, "count": 1 }
  },
  "items": [...]
}

```

7.6 GET /api/intents

```

Request :
GET /api/intents?lang=fr

Response :
{
  "total": 27,
  "intents": [
    {
      "tag": "inscription",
      "category": "inscription",
      "icon": "■",
      "nb_patterns": 12,

```

```
    "nb_responses": 3,  
    "example": "Comment s'inscrire ?"  
  },  
  ...  
]  
}
```


Chapitre 8 - Endpoints de l'academic-service

8.1 GET /api/overview (dashboard)

Response :

```
{
  "departments": 5,
  "filieres": 25,
  "modules": 100,
  "professors": 107,
  "students": 2970,
  "events": 5,
  "announcements": 5,
  "clubs": 8,
  "filieres_by_type": {
    "LICENCE": 6, "LICENCE_PRO": 1,
    "MASTER": 16, "MASTER_RECHERCHE": 2
  }
}
```

8.2 GET /api/filieres

Query params :

```
type           - LICENCE, LICENCE_PRO, MASTER, MASTER_RECHERCHE
department_id  - 1, 2, 3, 4, 5
is_active      - true / false
search         - texte a chercher dans nom ou code
```

Exemple :

```
GET /api/filieres?type=MASTER&search=info
```

Response :

```
[
  {
    "id": 8,
    "code": "MIAGE",
    "name": "Master Informatique et Gestion d'Entreprise",
    "type": "MASTER",
    "department_id": 1,
    "capacity": 30,
    "is_active": true
  },
  ...
]
```

8.3 GET /api/filieres/code/{code}

GET /api/filieres/code/IADS

Response :

```
{
  "id": 10,
  "code": "IADS",
```

```

    "name": "Master IA et Data Science",
    "type": "MASTER",
    "department_id": 1,
    "department_name": "Departement de Math-Info",
    "coordinator": "Pr. Mehdi FILALI",
    "modules_count": 13,
    "students_count": 48,
    ...
  }

```

8.4 GET /api/filieres/{id}/modules

GET /api/filieres/1/modules

Response :

```

{
  "filiere_id": 1,
  "total_modules": 30,
  "by_semester": [
    { "semester": 1, "modules": [
      { "code": "SMI-S1-M1", "name": "Analyse I",
        "credits": 6, "hours_cours": 36 },
      ...
    ]},
    { "semester": 2, "modules": [...]}],
    ...
  ]
}

```

8.5 GET /api/professors (paginé)

GET /api/professors?department_id=1&page=1&page_size=20

Response :

```

{
  "items": [
    { "id": 1, "matricule": "PROF100001",
      "first_name": "Mohammed", "last_name": "ALAOUI",
      "email": "...@fsbm.ac.ma", "grade": "PES",
      "specialty": "Intelligence Artificielle" },
    ...
  ],
  "total": 35,
  "page": 1,
  "page_size": 20,
  "total_pages": 2
}

```

8.6 Liste complete des endpoints academic

Methode	URL	Description
GET	/api/overview	Compteurs dashboard

GET	/api/departments	Liste des 5 dept
GET	/api/departments/{id}	Detail dept
GET	/api/departments/{id}/filieres	Filieres du dept
GET	/api/departments/{id}/professors	Profs du dept
GET	/api/filieres	Liste filieres + filtres
GET	/api/filieres/{id}	Detail filiere
GET	/api/filieres/code/{code}	Filiere par code
GET	/api/filieres/{id}/modules	Modules par semestre
GET	/api/modules	Liste modules + filtres
GET	/api/modules/{id}	Detail module
GET	/api/professors	Profs paginé
GET	/api/professors/{id}	Detail prof
GET	/api/students	Etudiants paginé
GET	/api/students/{id}	Detail etudiant
GET	/api/students/cne/{cne}	Etudiant par CNE
GET	/api/students/stats/by-filiere	Stats par filiere
GET	/api/schedule	Emploi du temps
GET	/api/exams	Calendrier examens
GET	/api/announcements	Annonces
GET	/api/events	Evenements
GET	/api/clubs	Clubs etudiants
GET	/api/health	Status service

Chapitre 9 - Endpoints LLM

9.1 POST /api/llm/chat

```
Request :
POST /api/llm/chat
{
  "message": "Quelles conditions pour le master IADS ?",
  "session_id": "sess_abc",
  "language": null,
  "temperature": 0.6
}

Response :
{
  "response": "Pour le master IADS, vous devez avoir...",
  "provider": "groq",
  "model": "llama-3.3-70b-versatile",
  "intent_detected": "master_iads",
  "confidence": 0.85,
  "language": "fr",
  "contexts_used": [
    { "tag": "master_iads", "score": 0.91, ... },
    { "tag": "masters", "score": 0.45, ... }
  ],
  "conversation_id": 42,
  "session_id": "sess_abc",
  "history_length": 4,
  "latency_ms": 187,
  "tokens_used": 423
}
```

9.2 GET /api/llm/status

```
Response :
{
  "groq": {
    "available": true,
    "model": "llama-3.3-70b-versatile"
  },
  "hf": {
    "available": false,
    "model": "meta-llama/Meta-Llama-3-8B-Instruct"
  },
  "fallback_tfidf": true,
  "primary": "groq"
}
```

9.3 GET /api/llm/models

```
Response :
{
  "groq": {
    "llama-large": "llama-3.3-70b-versatile",
    "llama-fast": "llama-3.1-8b-instant",
    "mixtral": "mixtral-8x7b-32768"
  }
}
```

```
    },  
    "huggingface": {  
      "llama": "meta-llama/Meta-Llama-3-8B-Instruct",  
      "mistral": "mistralai/Mistral-7B-Instruct-v0.3"  
    }  
  }  
}
```

Chapitre 10 - Communication inter-services

10.1 Pourquoi inter-services ?

Dans une architecture micro-services, les services doivent **collaborer**. Exemple : le chatbot-service a besoin des annonces stockees dans academic-service pour repondre a 'Cherche les dernieres news'.

10.2 Methodes possibles

Methode	Description	Utilisation
HTTP REST	Service A appelle service B	Notre choix (simple)
gRPC	RPC binaire performant	Microservices haute perf
Message queue	RabbitMQ, Kafka	Async + decouplage
GraphQL	API flexible avec query	BFF, agregation

10.3 Notre web_fetcher.py

Le chatbot-service appelle academic-service via HTTP avec httpx :

```
import httpx

class WebFetcher:
    def __init__(self, academic_service_url='http://localhost:8002'):
        self.academic_service_url = academic_service_url

    async def _fetch_from_academic_service(self):
        async with httpx.AsyncClient(timeout=5.0) as client:
            # 1. Recuperer les annonces
            r = await client.get(
                f'{self.academic_service_url}/api/annoncements',
                params={'limit': 5}
            )
            if r.status_code == 200:
                announcements = r.json()

            # 2. Recuperer les evenements
            r = await client.get(
                f'{self.academic_service_url}/api/events',
                params={'upcoming_only': True}
            )
            if r.status_code == 200:
                events = r.json()

        return announcements + events
```

10.4 Gestion des erreurs inter-services

```
try:
    async with httpx.AsyncClient(timeout=5.0) as client:
        r = await client.get(url)
        r.raise_for_status() # leve si 4xx ou 5xx
        return r.json()
except httpx.TimeoutException:
    # Service trop lent ou down
    return [] # mode degrade
except httpx.HTTPStatusError as e:
    # 4xx/5xx
    logger.error(f'Service {url} retournee {e.response.status_code}')
    return []
except Exception as e:
    logger.error(f'Erreur inattendue : {e}')
    return []
```

10.5 Cache TTL

Pour eviter de marteler academic-service, on cache les resultats 5 minutes :

```
class WebFetcher:
    CACHE_TTL = 5 * 60 # 5 minutes

    def _get_cached(self, key):
        if key in self._cache:
            entry = self._cache[key]
            if time.time() - entry.fetched_at < self.CACHE_TTL:
                return entry.items
        return None

    def _set_cached(self, key, items):
        self._cache[key] = CachedResult(items, time.time())
```

Chapitre 11 - Proxy Angular (proxy.conf.json)

11.1 Probleme : CORS

Le frontend Angular tourne sur port 4200. Les services backend tournent sur 8001 et 8002. Faire des requetes inter-ports declencherait CORS si pas configure.

11.2 Solution : proxy dev server

Angular CLI offre un **proxy integrate** au dev server. Le navigateur croit que tout vient de localhost:4200, mais en realite le serveur Angular redirige vers le bon backend.

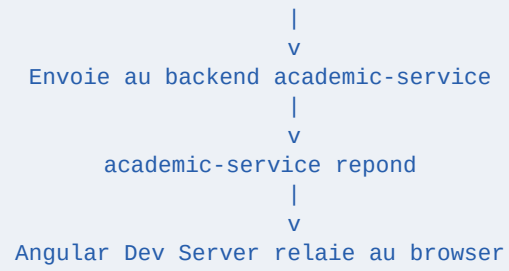
11.3 proxy.conf.json

```
{
  "/api/chat/feedback": {
    "target": "http://localhost:8001",
    "secure": false,
    "changeOrigin": true
  },
  "/api/chat": {
    "target": "http://localhost:8001",
    "secure": false,
    "changeOrigin": true
  },
  "/api/llm": {
    "target": "http://localhost:8001",
    "secure": false,
    "changeOrigin": true
  },
  "/api/academic": {
    "target": "http://localhost:8002",
    "secure": false,
    "changeOrigin": true,
    "pathRewrite": { "^/api/academic": "/api" }
  }
}
```

11.4 Comment ca marche

```

      Browser fait :
GET http://localhost:4200/api/academic/filieres
      |
      v
    Angular Dev Server intercepte
      |
      v
    Consulte proxy.conf.json
    Trouve /api/academic -> 8002 + pathRewrite
      |
      v
    Reecrit URL : http://localhost:8002/api/filieres
  
```

[★] En production, on remplace ce proxy par un **reverse proxy nginx** qui fait pareil. Le frontend devient un site statique servi par nginx, qui redirige /api/... vers les services backend.

Chapitre 12 - Swagger/OpenAPI

12.1 OpenAPI specification

OpenAPI (anciennement Swagger) est une specification standard pour decire des APIs REST. Format YAML ou JSON. FastAPI genere cette specification automatiquement.

12.2 Acces dans notre projet

- <http://localhost:8001/docs> - Swagger UI interactive
- <http://localhost:8001/redoc> - ReDoc (plus belle, non-interactive)
- <http://localhost:8001/openapi.json> - spec OpenAPI JSON

12.3 Ce qu'on voit sur /docs

- Liste de TOUS les endpoints groupes par tag
- Pour chaque endpoint : methode, URL, description
- Schemas Pydantic des bodies (request + response)
- Codes de statut possibles
- Bouton **Try it out** pour tester en direct
- Generation automatique de curl

12.4 Ameliorer la doc dans le code

```
@router.post(
    '/chat',
    response_model=ChatResponse,
    summary='Envoyer un message au chatbot',
    description=''
    Le chatbot detecte la langue, classifie l'intent et genere une reponse.

    Memoire conversationnelle 10 tours. Personnalisation genre/nom.
    '',
    tags=['chat'],
    responses={
        200: { 'description': 'Reponse OK' },
        422: { 'description': 'Validation echouee' },
        500: { 'description': 'Erreur serveur' },
    }
)
async def chat(req: ChatRequest):
    ...
```

Chapitre 13 - Tester les APIs

13.1 Avec curl (ligne de commande)

```
# GET simple
curl http://localhost:8001/api/health

# POST avec body JSON
curl -X POST http://localhost:8001/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Bonjour"}'

# Sauver la reponse dans un fichier
curl http://localhost:8002/api/filieres -o filieres.json
```

13.2 Avec Postman

Postman est une app GUI gratuite pour tester les APIs. Tu peux :

- Sauvegarder une collection de requetes
- Definir des variables d'environnement
- Ecrire des tests automatises (JavaScript)
- Partager avec ton equipe
- Generer la doc auto

13.3 Avec Swagger UI

Le plus simple pour notre demo : <http://localhost:8001/docs> et <http://localhost:8002/docs>. Clique sur un endpoint, 'Try it out', remplis le body, 'Execute'. La reponse s'affiche.

13.4 Avec Python (requests / httpx)

```
import httpx

# Async
async with httpx.AsyncClient() as client:
    response = await client.post(
        'http://localhost:8001/api/chat',
        json={'message': 'Bonjour'}
    )
    print(response.status_code)
    print(response.json())

# Sync (avec requests)
import requests
r = requests.post(
    'http://localhost:8001/api/chat',
    json={'message': 'Bonjour'}
)
print(r.json())
```

Chapitre 14 - Conclusion

14.1 Recap

- + HTTP = protocole client-serveur en text
- + Requete = methode + URL + headers + body
- + Reponse = code statut + headers + body
- + 5 methodes principales : GET/POST/PUT/PATCH/DELETE
- + 16 codes de statut essentiels (200, 404, 500...)
- + Headers pour metadonnees (Content-Type, Authorization, CORS...)
- + JSON = format standard d'echange
- + REST = principes de design (stateless, uniform interface)
- + 23 endpoints academic + 8 endpoints chatbot + 3 endpoints LLM
- + Communication inter-services via HTTP REST avec httpx async
- + Proxy Angular pour eviter CORS en dev
- + Swagger UI auto sur /docs

14.2 Pour aller plus loin

- PDF 03 - implementation FastAPI cote serveur
- PDF 08 - securite des APIs (JWT, CORS, etc.)
- PDF 09 - workflow complet incluant les appels API